**JAVASCRIPT + DOM**

# Course Reference

Part I - Foundations to Data + DOM Integration

**How to use this reference**
This is not meant to be reread from page 1. Use the Quick-Look Map first, then jump to the module you need. Each section gives the purpose, syntax, grammar, data form, and the connection to the rest of the program.

*Completed material only - next new chapter: Functions*

# Quick-Look Master Map

```
JAVASCRIPT
|
+-- DATA
|   +-- string       "Maomao"
|   +-- number       9
|   +-- boolean      true / false
|   +-- array        ["Maomao", "Mangos"]
|
+-- MEMORY / LOGIC
|   +-- const / let
|   +-- if / else
|   +-- ===
|   +-- !
|
+-- REPEAT
|   +-- for...of
|
+-- EVENTS
|   +-- addEventListener()
|
+-- DOM
    +-- FIND       getElementById()
    +-- CREATE     createElement()
    +-- CHANGE     textContent / innerHTML / style / classList
    +-- ADD        appendChild()
    +-- REMOVE     remove()
```

## The Big Picture

```
SOURCE OF DATA
(user / array / later: API, storage, time, etc.)
        |
        v
    JAVASCRIPT
        |
        +-- variables hold things
        +-- logic decides things
        +-- loops repeat things
        +-- events decide WHEN things happen
        |
        v
      DOM APIs
        |
        v
 HTML PAGE CHANGES
```

**Most important connection**

Do not memorize programs as mysterious lines. Follow the data: Where did the value come from? What form is it? Where does it go next?

# Module 1 - JavaScript Talks to HTML

## 1.1 Finding an HTML Element

**Purpose:** JavaScript must locate an existing HTML element before it can change it.

```
const message = document.getElementById("message");
```

**Read it:** Find the HTML element whose ID is "message" and store that element as message.

**Form / type:** message -> HTML ELEMENT

- document = browser document object
- . = connect object to property/method
- getElementById() = method
- "message" = string argument

## 1.2 Changing Text

**Purpose:** Change the plain text inside an HTML element.

```
message.textContent = "Hello";
```

**Read it:** Set the text content of message to "Hello".

- .textContent = property
- = = assign
- "Hello" = string

## 1.3 Changing HTML

**Purpose:** Insert HTML markup rather than plain text.

```
message.innerHTML = "<strong>Hello</strong>";
```

- textContent -> plain text
- innerHTML -> HTML markup

## 1.4 Changing Style

**Purpose:** Change CSS through JavaScript.

```
message.style.color = "red";
message.style.backgroundColor = "black";
message.style.fontSize = "20px";
message.style.display = "none";
```

- Pattern: element.style.property = value;

- CSS background-color becomes JavaScript backgroundColor.

### 1.5 Changing Classes

**Purpose:** Add or remove CSS classes.

```
message.classList.add("active");
message.classList.remove("active");
```

- classList is a property.
- add() and remove() are methods/actions.

# Module 2 - Creating and Removing HTML

### 2.1 Creating an Element

**Purpose:** Create a new HTML element in JavaScript.

```
const newItem = document.createElement("p");
```

**Read it:** Create a paragraph element and store it as newItem.

**Form / type:** newItem -> HTML ELEMENT 

> **Important**
> Creating an element does not automatically make it visible on the webpage. It must later be attached to the DOM.

### 2.2 Giving It Content

**Purpose:** Put text inside the newly created element.

```
newItem.textContent = "Hello";
```

-  -> Hello

### 2.3 Attaching It to the Page

**Purpose:** Put a new element inside an existing parent/container.

```
container.appendChild(newItem);
```

**Read it:** Append newItem as a child of container.

- container -> parent HTML element

- newItem -> child HTML element

### 2.4 Removing an Element

**Purpose:** Remove an HTML element.

```
newItem.remove();
```

- remove() is a method/action, so it needs parentheses.

# Module 3 - Events: WHEN JavaScript Runs

### 3.1 Event Listeners

**Purpose:** Wait for an event before running instructions.

```
button.addEventListener("click", function () {
    // instructions
});
```

- "click" = event being watched
- function () { } = instructions that run after the event
- }); closes the function block and the method call.

```
USER CLICKS
     |
     v
 "click" event
     |
     v
 function runs
     |
     v
 instructions happen
```

# Module 4 - JavaScript Logic and Memory

### 4.1 Variables

**Purpose:** Store values for JavaScript to use.

```
const city = "Qingdao";
let catAwake = false;
```

- const -> not reassigned later
- let -> can be reassigned later

### 4.2 Booleans

**Purpose:** Represent two-state information.

```
true
false

let catAwake = false;
```

- Useful for states such as on/off, open/closed, awake/asleep.

### 4.3 Flipping a Boolean

**Purpose:** Switch a boolean to its opposite.

```
catAwake = !catAwake;
```

- false -> true
- true -> false
- ! means NOT and flips a boolean in this pattern.

### 4.4 Conditions

**Purpose:** Run different instructions depending on a condition.

```
if (catAwake === true) {
    // do this
} else {
    // do something else
}
```

- else if adds another condition.
- = assigns a value.
- === compares values.

### 4.5 Console

**Purpose:** Inspect information without putting it on the webpage.

```
console.log(catAwake);
```

- Output appears in the browser developer console.
- Useful for checking values and debugging.

# Module 5 - JavaScript Grammar Reference

**Use this module as a lookup page**

This is a grammar reference, not a lesson you need to reread in order.

```
object.property
object.property = value;

object.method();
object.method(value);
```

| Symbol | Meaning |
| --- | --- |
| . | connect object -> property/method |
| () | call/run something |
| {} | block of instructions |
| [] | array / array access |
| "" | string |
| = | assign |
| === | compare |
| ! | NOT / flip boolean |
| ; | end statement |

## Property vs Method

```
PROPERTY = information
cityInput.value
cats.length
message.textContent

METHOD = action
message.remove()
cats.push("Luna")
document.createElement("p")
```

## camelCase and Case Sensitivity

```
getElementById
createElement
textContent
appendChild
addEventListener
```

- JavaScript is case-sensitive.
- Your own names often use camelCase by convention.
- Built-in API capitalization is fixed: createElement is correct; createelement is not.

# Module 6 - User Input

## 6.1 HTML Input

**Purpose:** Create a box where a user can type.

```
<input id="cityInput" placeholder="Enter your city">
```

## 6.2 Reading the Input

**Purpose:** Read the text currently typed into an input.

```
const cityInput = document.getElementById("cityInput");

cityInput.value
```

- If the user typed Qingdao, cityInput.value contains "Qingdao".
- .value is a property.

## 6.3 Timing Matters

**Purpose:** Read the value after the user has had a chance to type.

```
button.addEventListener("click", function () {
    const city = cityInput.value;
});
```

- If you read .value too early, the input may still be empty.

```
user types
    |
    v
user clicks
    |
    v
event runs
    |
    v
JavaScript reads .value
```

**Important distinction**
HTML <input> is one source of data, not the definition of data. JavaScript data can also come from arrays and, later in the course, APIs, localStorage, Date(), Math.random(), and other sources.

# Module 7 - Arrays

## 7.1 Creating an Array

**Purpose:** Store a collection of related values.

```
const cats = ["Maomao", "Mangos"];
```

**Form / type:** cats -> ARRAY ["Maomao", "Mangos"]

## 7.2 Indexes

**Purpose:** Access one item by position.

```
const cats = ["Maomao", "Mangos", "Luna"];
cats[0]  // "Maomao"
```

- Indexes start at 0: Maomao=0, Mangos=1, Luna=2.

## 7.3 Array Length

**Purpose:** Find how many items an array contains.

```
cats.length
```

- .length is a property, not a method.

## 7.4 Adding an Item

**Purpose:** Add an item to the end of an array.

```
cats.push("Luna");
```

- push() is a method/action.

## 7.5 Removing the Last Item

**Purpose:** Remove the final array item; optionally store what was removed.

```
cats.pop();

const removedCat = cats.pop();
```

### 7.6 Changing an Item

**Purpose:** Replace a value at a particular index.

```
cities[2] = "Vancouver";
```

# Module 8 - Loops

### 8.1 Why Loops Exist

**Purpose:** Repeat the same instructions for multiple values without writing the instructions repeatedly.

### 8.2 for...of

**Purpose:** Take one value from an array at a time and run the block for that value.

```
for (const cat of cats) {
    console.log(cat);
}
```

**Read it:** For each value in cats, temporarily call that value cat and run these instructions.

```
cats -> WHOLE ARRAY

1st trip -> cat = "Maomao"
2nd trip -> cat = "Mangos"
3rd trip -> cat = "Luna"
```

**The connection that was easy to miss**
There is no visible = in 'for (const cat of cats)', but the loop supplies cat with one value from the array on each trip.

# Module 9 - Data Forms and Data Flow

**Why this module matters**
This is the bridge between knowing individual commands and understanding a whole program.

### 9.1 Different Variables Can Hold Different Forms

**Purpose:** Recognize what kind of thing a variable currently refers to.

```
const cat = "Maomao";
const cats = ["Maomao", "Mangos"];
```

```
const paragraph = document.createElement("p");
```

```
cat       -> "Maomao"                STRING
cats      -> ["Maomao", "Mangos"]    ARRAY
paragraph -> <p></p>                  HTML ELEMENT
```

### 9.2 A Value Can Travel Into Another Structure

**Purpose:** Follow data as it is used by another object.

```
paragraph.textContent = cat;
```

```
"Maomao"  STRING
    |
    v
paragraph.textContent
    |
    v
<p>Maomao</p>  HTML ELEMENT
```

- cat and paragraph do not become each other.
- The string value stored in cat is used as the paragraph's text.

### 9.3 Existing vs Newly Created Elements

**Purpose:** Distinguish an element already in the page from a new element created by JavaScript.

```
const catList = document.getElementById("catList");
const newCat = document.createElement("p");
catList.appendChild(newCat);
```

- catList -> existing HTML container
- newCat -> newly created HTML element
- appendChild() puts the new element inside the existing container.

## Module 10 - Array + Loop + DOM Integration

```
const cats = ["Maomao", "Mangos"];

const catList = document.getElementById("catList");

for (const cat of cats) {
    const newCat = document.createElement("p");
    newCat.textContent = cat;
    catList.appendChild(newCat);
}
```

### Read the Journey, Not Just the Lines

```
ARRAY
["Maomao", "Mangos"]
        |
        v
LOOP takes one value
        |
        v
STRING
"Maomao"
        |
        v
createElement("p")
        |
        v
HTML ELEMENT
<p></p>
        |
        v
.textContent = cat
        |
        v
<p>Maomao</p>
        |
        v
appendChild()
        |
        v
HTML CONTAINER
        |
        v
WEBPAGE
```

**Chapter stopping point**
You can now connect an array to a loop, take one string at a time, create an HTML element, put the string into it, and attach it to the webpage. This completes Part I.

# Next Chapter - Functions

Next we will learn reusable functions, parameters, arguments, and return values. The same data-flow method will be used: where a value comes from, what form it has, and where it goes next.

```
function addItem(text) {
    ...
}
```